

Lab Task: Implement A* Search on Romanian Map

Your goal is to implement the *A* search algorithm* to find the shortest path from **Arad** to **Bucharest** on the Romanian map.

You are given:

1. **Graph of Romanian cities and roads (Fig. 3.1):** Each city is a node. Each road between cities has a distance (cost). The graph is undirected (roads go both ways). You can represent this graph using a suitable data structure.

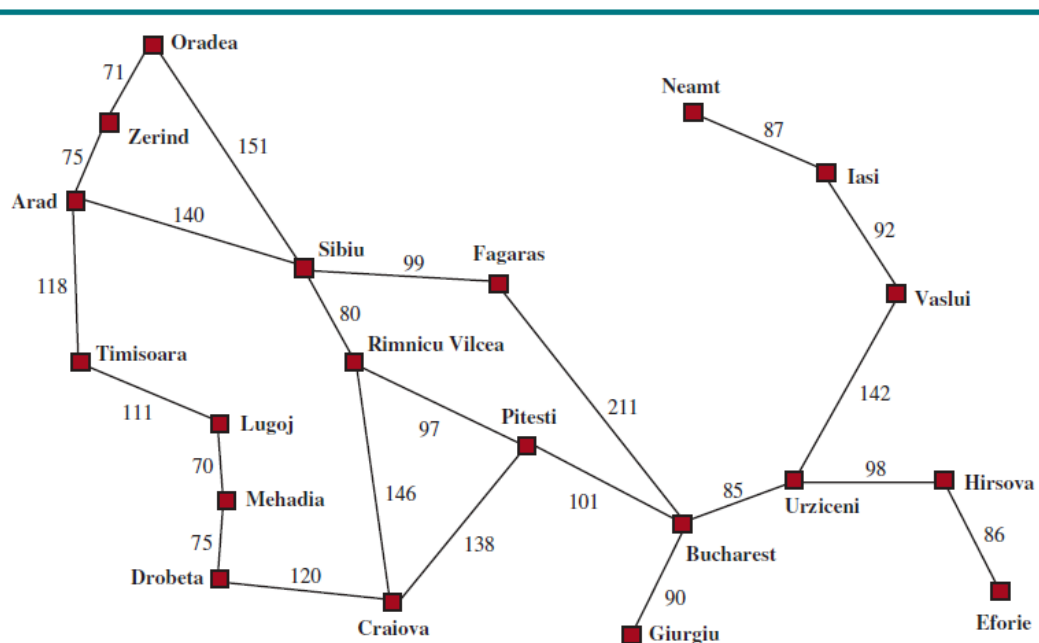


Figure 3.1 A simplified road map of part of Romania, with road distances in miles.

2. **Heuristic values (straight-line distances) (Fig. 3.16):** For each city, you have the estimated straight-line distance to Bucharest (the goal).

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.16 Values of h_{SLD} —straight-line distances to Bucharest.

See the visualization for *Stages in an A* search for Bucharest* from Figure 3.18 in the textbook if needed.

What you need to design and implement:

1. Data structures

- How will you store the graph and the heuristic values? (Hint: dictionaries can be very useful.)
- What data structure will you use to keep track of nodes to explore next, so you always pick the best one? (Hint: priority queue or min-heap)

2. Algorithm

- Start at Arad (initial node).
- Keep track of the path cost so far ($g(n)$) for each city you reach.
- Use the heuristic ($h(n)$) to estimate the remaining cost to Bucharest.
- Calculate the total estimated cost ($f(n) = g(n) + h(n)$) for each city in the frontier.
- Always expand the city with the lowest $f(n)$.
- Track the parent of each city so you can reconstruct the final path once you reach Bucharest.
- For cases where you are not given the heuristic values, you need to calculate path distances from any city to any other city. In that case, you will be provided with the (x, y) coordinates of each city, and you must dynamically compute the Manhattan distance using the formula:

$$\text{Manhattan Distance} = |x_1 - x_2| + |y_1 - y_2|$$

This distance will serve as your heuristic function in the A* search algorithm.

3. Output

- a. Once Bucharest is reached, reconstruct and print the path from Arad to Bucharest.
- b. Print the total cost of the path.
- c. Print the state of the map after each step (as in Figure 3.18 in the textbook).
- d. Extend the code to find paths from any city to Bucharest by accepting user input. Then further extend it to find paths between any two cities based on user input.

Hints:

1. Use a priority queue to efficiently pick the next city with the lowest estimated total cost.
2. Use a dictionary to remember the cheapest known cost to each city.
3. Use another dictionary to remember from which city you came to reconstruct the path later.
4. When you find a better path to a city, update the cost and parent info and add it to the frontier again.

Deliverables:

1. Your Python code implementing A* search.
2. Example output showing the shortest path from Arad to Bucharest and the total path cost.
3. A short explanation of how your code works and which data structures you used.